Open in Colab YouTube

##1. Introduction to Association Rule Mining - Market Baset Analysis (MBA)

- Association rule learning is a rule-based (if-then-else) machine learning method for discovering interesting relations between variables in large databases.
- It is intended to identify strong rules discovered in databases using some measures of interestingness
- In any given transaction with a variety of items, association rules are meant to discover the rules that determine how or why certain items are connected.
- Market Basket Analysis (MBA) is a specific case of Association Rule Mining, hence, it is a data mining technique used to discover patterns and relationships between items purchased together in transactions.
- It helps identify combinations of products that are frequently bought together, providing valuable insights for businesses to optimize sales, marketing, and inventory strategies.
- Based on the concept of strong rules, Rakesh Agrawal, Tomasz Imieliński and Arun Swami introduced association rules for discovering regularities between products in large-scale transaction data recorded by point-of-sale (POS) systems in supermarkets.

For example, the rule:

$$\{\text{onions}, \text{potatoes}\} \Rightarrow \{\text{burger}\}$$

found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, they are likely to also buy hamburger meat. Such information can be used as the basis for decisions about marketing activities such as, e.g., promotional pricing or product placements.

In contrast with sequence mining, association rule learning typically does not consider the order of items either within a transaction or across transactions.

reference: https://en.wikipedia.org/wiki/Association_rule_learning

---

Key Concepts:

1. Association Rules:

- These are if-then rules that identify relationships between items. For example, "If a customer buys bread, they are likely to buy butter."

2. Metrics:

2.1. Support:

Measures how often an item set appears in the dataset.
$$ \text{Support(A)} = \frac{\text{Number of transactions containing A}}{\text{Total transactions}} $$

Valid Values for Support:

- **Range**: The value of support ranges from **0% to 100%**:
- **0%**: The itemset does not appear in any transaction.
- **100%**: The itemset appears in all transactions.
- *Interpreting Support:**
- **High support**: The itemset is common and may represent a strong association.
- **Low support**: The itemset is rare and might not provide actionable insights.
- *Choosing Support Thresholds**
- In practice, a minimum support threshold is specified to filter out infrequent itemsets, ensuring only itemsets that appear frequently in the dataset are considered for further analysis.

- Setting a very high support threshold can exclude rare but potentially interesting itemsets.
- A very low support threshold might lead to too many itemsets, making the analysis computationally expensive and cluttered.

2.2. Confidence:

Measures how often the rule is true.

$$\text{Confidence}(\text{A} \rightarrow \text{B}) = \frac{\text{Support}(\text{A} \cap \text{B})}{\text{Support}(\text{A})}$$

Valid Values for Confidence

- **Range**: Confidence values range from **0 to 1**:
  - **0**: The rule $X \rightarrow Y$ never holds true.
  - **1**: The rule $X \rightarrow Y$ always holds true (perfect confidence).

Interpreting Confidence

1. **High Confidence (≈ 1)**:
   - Indicates that Y almost always occurs when X occurs.
   - Example: If $X = \{\text{bread}\}$ and $Y = \{\text{butter}\}$, a high confidence implies that customers buying bread are very likely to buy butter.
2. **Low Confidence (≈ 0)**:
   - Indicates that Y rarely occurs when X occurs.
   - Example: If $X = \{\text{bread}\}$ and $Y = \{\text{cereal}\}$, a low confidence implies that buying bread is not strongly associated with buying cereal.

Confidence vs. Other Metrics

- **Confidence** does not consider how common Y is overall. This can lead to misleading rules if Y is very frequent in the dataset. For example, a high-confidence rule might occur simply because Y is common, not because X and Y are strongly related.
- To address this, metrics like **lift** or **conviction** are often used alongside confidence.

Thresholds in Practice

- A **minimum confidence threshold** is typically set to filter rules that meet a desired level of predictive strength.
- A common approach is to start with rules that have both high confidence (e.g., $\text{Confidence} > 0.7$) and high support.

2.3. Lift:

Measures how much more likely two items are to be bought together compared to being independent.

$$\text{Lift}(\text{A} \rightarrow \text{B}) = \frac{\text{Confidence}(\text{A} \rightarrow \text{B})}{\text{Support}(\text{B})}$$

In the Apriori algorithm, **lift** is a metric used to evaluate the strength of a rule in association rule mining. It is calculated as:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Support}(X \cap Y)}{\text{Support}(X) \times \text{Support}(Y)}$$

Valid Values for Lift

1. **Greater than 1**:
   - Indicates a **positive association** between X and Y.
   - X and Y are more likely to occur together than if they were independent.
2. **Equal to 1**:
   - Indicates **no association** between X and Y.
   - X and Y occur together just as often as would be expected under independence.
3. **Less than 1**:
   - Indicates a **negative association** between X and Y.
   - X and Y are less likely to occur together than if they were independent.

When using the Apriori algorithm, a lift threshold is often set to filter rules that meet a certain level of interest, typically focusing on those with $\text{Lift} > 1$.

Association Rule Mining Domains Other Than Market Basket Analysis

Association rule mining can be applied in various domains beyond market basket analysis. Here are some examples:

1. **Healthcare**
   - **Disease Diagnosis:** Identifying associations between symptoms and diseases. For example, a rule like *{fever, cough} → {flu}* can help predict diseases.
   - **Drug Interactions:** Discovering relationships between medications that are frequently prescribed together or identifying combinations that lead to adverse reactions.
2. **Web Usage Mining**
   - **Website Optimization:** Analyzing user navigation patterns to determine common paths or clicks, e.g., *{homepage → product page} → {checkout}*.
   - **Recommendation Systems:** Suggesting content or products based on frequently co-accessed items, e.g., *{clicked 'smartphone'} → {clicked 'smartphone accessories'}*.
3. **Education**
   - **Student Behavior Analysis:** Discovering patterns in course enrollment, such as *{math101, cs101} → {stat101}*.
   - **Learning Paths:** Identifying sequences of topics that students study, helping design better curricula.
4. **Telecommunications**
   - **Customer Churn Analysis:** Detecting combinations of usage patterns that are associated with customers leaving the service, e.g., *{low data usage, few calls} → {churn}*.
   - **Service Bundling:** Identifying services that are commonly purchased together, like *{broadband, mobile} → {TV subscription}*.
5. **Banking and Finance**
   - **Fraud Detection:** Uncovering patterns associated with fraudulent transactions, e.g., *{high transaction frequency, odd hours} → {fraud}*.
   - **Loan Approvals:** Identifying attributes of successful loan applications, such as *{high income, good credit score} → {loan approved}*.
6. **Manufacturing**
   - **Fault Detection:** Identifying combinations of machine conditions that frequently result in faults, e.g., *{high temperature, low pressure} → {equipment failure}*.
   - **Supply Chain Optimization:** Discovering patterns in material usage, e.g., *{material A, material B} → {product C}*.
7. **Retail Beyond Market Basket**
   - **Shelf Placement:** Finding products that are often bought together to optimize store layout.
   - **Customer Segmentation:** Identifying customer groups with similar purchasing behaviors, e.g., *{frequent discount purchases} → {low brand loyalty}*.
8. **Social Media Analysis**
   - **Trending Topics:** Discovering associations between hashtags, e.g., *{#climatechange, #sustainability} → {#renewableenergy}*.
   - **User Behavior Patterns:** Understanding engagement behaviors, such as *{likes post, comments on post} → {shares post}*.
9. **Energy Sector**
   - **Usage Patterns:** Identifying associations in energy usage, like *{high A/C usage, weekend} → {peak energy consumption}*.
   - **Smart Grid Analysis:** Detecting patterns for predictive maintenance, e.g., *{low voltage, high demand} → {power outage}*.
10. **Transportation and Logistics**
    - **Traffic Analysis:** Discovering patterns in traffic conditions, e.g., *{morning rush hour, bad weather} → {traffic jam}*.
    - **Route Optimization:** Identifying frequently used delivery routes, such as *{route A, route B} → {delivered faster}*.
11. **E-commerce**
    - **User Preferences:** Identifying patterns in user preferences for personalized recommendations.
    - **Cross-Selling:** Suggesting related products based on purchase history.
12. **Sports Analytics**
    - **Performance Metrics:** Discovering combinations of player actions that lead to victories, e.g., *{high possession, accurate passes} → {win}*.
    - **Injury Prevention:** Identifying conditions that precede injuries, such as *{high training load, lack of rest} → {injury risk}*.

---

3. **Apriori Algorithm**:

This is the most common algorithm used for MBA. It identifies frequent itemsets and builds association rules from them by leveraging the principle that subsets of frequent itemsets must also be frequent.

Step-by-Step Execution of the Apriori Algorithm

We will go step-by-step through how the **Apriori algorithm** would process this dataset. We'll compute frequent itemsets and generate rules using the given data, along with thresholds for **support**, **confidence**, and **lift**.

Let's assume a small dataset with 12 transactions:

Transaction ID	Item 1	Item 2	Item 3	Item 4	Item 5
1	Milk	Egg	Bread	Butter	
2	Milk	Butter	Egg	Ketchup	
3	Bread	Butter	Ketchup		
4	Milk	Bread	Butter		
5	Bread	Butter	Cookies		
6	Milk	Bread	Butter	Cookies	
7	Milk	Cookies			
8	Milk	Bread	Butter		
9	Bread	Butter	Egg	Cookies	
10	Milk	Butter	Bread		
11	Milk	Bread	Butter		
12	Milk	Bread	Cookies	Ketchup	

We set out thresholds as following:

- **Minimum Support**: 50% (focus on items present in at least half of the transactions).
- **Minimum Confidence**: 70%.
- **Lift**: Lift > 1 for actionable rules.

Step 1: Dataset Transformation Convert the dataset into a binary format:

Transaction	Milk	Egg	Bread	Butter	Ketchup	Cookies
1	1	1	1	1	0	0
2	1	1	0	1	1	0
3	0	0	1	1	1	0
4	1	0	1	1	0	0
5	0	0	1	1	0	1
6	1	0	1	1	0	1
7	1	0	0	0	0	1
8	1	0	1	1	0	0
9	0	1	1	1	0	1
10	1	0	1	1	0	0
11	1	0	1	1	0	0
12	1	0	1	0	1	1

Step 2: Identify Frequent 1-Itemsets The algorithm starts by calculating the support for each single item.

Item	Support Count	Support (%)	Frequent?
Milk	9	75%	Yes
Egg	3	25%	No
Bread	10	83%	Yes
Butter	10	83%	Yes
Ketchup	3	25%	No
Cookies	5	42%	No

Frequent 1-itemsets: `{Milk}, {Bread}, {Butter}`.

Step 3: Generate 2-Itemsets Now, combine frequent 1-itemsets into 2-itemsets and calculate their support.

Itemset	Support Count	Support (%)	Frequent?
{Milk, Bread}	7	58%	Yes

Itemset	Support Count	Support (%)	Frequent?
{Milk, Butter}	7	58%	Yes
{Bread, Butter}	9	75%	Yes

Frequent 2-itemsets: `{Milk, Bread}`, `{Milk, Butter}`, `{Bread, Butter}`.

Step 4: Generate 3-Itemsets Combine frequent 2-itemsets into 3-itemsets and calculate their support.

Itemset	Support Count	Support (%)	Frequent?
{Milk, Bread, Butter}	6	50%	Yes

Frequent 3-itemset: `{Milk, Bread, Butter}`.

Step 5: No Larger Itemsets Since there are no frequent 4-itemsets (support would drop below 50%), we stop here.

Step 6: Generate Association Rules

Now, use the frequent itemsets to generate association rules and calculate their metrics: **support**, **confidence**, and **lift**.

Rules from `{Milk, Bread, Butter}`:

1. **Rule**:
$$\text{Milk, Bread} \rightarrow \text{Butter}$$
   - **Support**:
$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$
   - **Confidence**:
$$P(\text{Butter}|\text{Milk, Bread}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Milk, Bread})} = \frac{6}{7} = 85$$
   - **Lift**:
$$\text{Lift} = \frac{P(\text{Butter}|\text{Milk, Bread})}{P(\text{Butter})} = \frac{0.85}{0.83} \approx 1.02$$
2. **Rule**:
$$\text{Milk, Butter} \rightarrow \text{Bread}$$
   - **Support**:
$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$
   - **Confidence**:
$$P(\text{Bread}|\text{Milk, Butter}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Milk, Butter})} = \frac{6}{7} = 85$$
   - **Lift**:
$$\text{Lift} = \frac{P(\text{Bread}|\text{Milk, Butter})}{P(\text{Bread})} = \frac{0.85}{0.83} \approx 1.02$$
3. **Rule**:
$$\text{Bread, Butter} \rightarrow \text{Milk}$$
   - **Support**:
$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$
   - **Confidence**:
$$P(\text{Milk}|\text{Bread, Butter}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Bread, Butter})} = \frac{6}{9} \approx 66$$
   - **Lift**:
$$\text{Lift} = \frac{P(\text{Milk}|\text{Bread, Butter})}{P(\text{Milk})} = \frac{0.66}{0.75} \approx .88$$

Final Results

Frequent Itemsets:

- **1-itemsets**: `{Milk}`, `{Bread}`, `{Butter}`
- **2-itemsets**: `{Milk, Bread}`, `{Milk, Butter}`, `{Bread, Butter}`
- **3-itemset**: `{Milk, Bread, Butter}`

Rules: | Rule | Support | Confidence | Lift | Actionable? | |----------|---------|----------|------|----------| |

$$\text{Milk, Bread} \rightarrow \text{Butter}$$

| 50% | 85% | 1.02 | Yes | |

$$\text{Milk, Butter} \rightarrow \text{Bread}$$

| 50% | 85% | 1.02 | Yes | |

$$\text{Bread, Butter} \rightarrow \text{Milk}$$

| 50% | 66% | .88 | No |

Interpretation and Actionable Insights | Rule | Explaniation | |----------|----------| |

$$\text{Milk, Bread} \rightarrow \text{Butter}$$

|Customers buying Milk and Bread are highly likely (85%) to also buy Butter. Consider bundling these items. | |

$$\text{Milk, Butter} \rightarrow \text{Bread}$$

| Strong association; placing these items together could increase sales. | |

$$\text{Bread, Butter} \rightarrow \text{Milk}$$

| Suggests that Milk is a complementary product to Bread and Butter, Weak lift thought |

---

3. **Python Code Example**

We will implement the **Apriori algorithm** using the `mlxtend` library for frequent itemset mining and rule generation.

a. Install Required Libraries Make sure you have the required libraries installed:

```
! pip install pandas mlxtend
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell`
  and should_run_async(code)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (2.2.2)
Requirement already satisfied: mlxtend in /usr/local/lib/python3.10/dist-packages (0.23.3)
Requirement already satisfied: numpy>=1.22.4 in /usr/local/lib/python3.10/dist-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas) (2024.2)
Requirement already satisfied: scipy>=1.2.1 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.13.1)
Requirement already satisfied: scikit-learn>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.6.0)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (3.8.0)
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (4.55.3)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (24.2)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (11.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.3.1->mlxtend) (3.5.
```

b. Load the Dataset We will represent the dataset as a binary **transaction matrix** where each row is a transaction, and each column is an item (1 = purchased, 0 = not purchased).

```
import pandas as pd

# Define the dataset
data = {
    "Milk":    [1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1],
    "Egg":     [1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
    "Bread":   [1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1],
    "Butter":  [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0],
    "Ketchup": [0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    "Cookies": [0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1],
}

# Create a DataFrame
df = pd.DataFrame(data)
print("Transaction Dataset:")
df
```

```
Transaction Dataset:
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell`
  and should_run_async(code)
```

	Milk	Egg	Bread	Butter	Ketchup	Cookies
0	1	1	1	1	0	0
1	1	1	0	1	1	0
2	0	0	1	1	1	0
3	1	0	1	1	0	0
4	0	0	1	1	0	1
5	1	0	1	1	0	1
6	1	0	0	0	0	1
7	1	0	1	1	0	0
8	0	1	1	1	0	1
9	1	0	1	1	0	0
10	1	0	1	1	0	0
11	1	0	1	0	1	1

Next steps: Generate code with df | View recommended plots | New interactive sheet

c. Apply the Apriori Algorithm We will use the `mlxtend` library to find frequent itemsets and generate association rules.

```
from mlxtend.frequent_patterns import apriori, association_rules

# Step 1: Generate frequent itemsets with Apriori
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

# Display frequent itemsets
print("\nFrequent Itemsets:")
print(frequent_itemsets)
```

```
Frequent Itemsets:
    support               itemsets
0  0.750000                 (Milk)
1  0.833333                (Bread)
2  0.833333               (Butter)
3  0.583333          (Bread, Milk)
4  0.583333         (Butter, Milk)
5  0.750000        (Bread, Butter)
6  0.500000  (Bread, Butter, Milk)
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell`
  and should_run_async(code)
/usr/local/lib/python3.10/dist-packages/mlxtend/frequent_patterns/fpcommon.py:161: DeprecationWarning: DataFrames with non-bool types re
  warnings.warn(
```

d. Generate Association Rules We generate rules based on **confidence** and calculate **lift**.

```
# Step 2: Generate association rules
rules = association_rules(frequent_itemsets, num_itemsets=len(df), metric="confidence", min_threshold=0.7)

# Display the rules
print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
Association Rules:
      antecedents consequents   support  confidence      lift
0         (Bread)      (Milk)  0.583333    0.700000  0.933333
1          (Milk)     (Bread)  0.583333    0.777778  0.933333
2        (Butter)      (Milk)  0.583333    0.700000  0.933333
3          (Milk)    (Butter)  0.583333    0.777778  0.933333
4         (Bread)    (Butter)  0.750000    0.900000  1.080000
5        (Butter)     (Bread)  0.750000    0.900000  1.080000
6   (Bread, Milk)    (Butter)  0.500000    0.857143  1.028571
7  (Butter, Milk)     (Bread)  0.500000    0.857143  1.028571
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell`
  and should_run_async(code)
```

e. Filter and Interpret Rules You can filter rules for high lift or specific itemsets.

```
# Filter rules with Lift > 1
filtered_rules = rules[rules['lift'] > 1]
print("\nFiltered Rules with Lift > 1:")
print(filtered_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
Filtered Rules with Lift > 1:
      antecedents consequents  support  confidence      lift
4         (Bread)    (Butter)     0.75    0.900000  1.080000
5        (Butter)     (Bread)     0.75    0.900000  1.080000
6   (Bread, Milk)    (Butter)     0.50    0.857143  1.028571
7  (Butter, Milk)     (Bread)     0.50    0.857143  1.028571
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell`
  and should_run_async(code)
```

Interpretation

1. $$\text{Milk, Bread} \rightarrow \text{Butter}$$
   - Support = 58%, Confidence = 87.5%, Lift = 1.05.
   - Suggest bundling these items for promotions.
2. $$\text{Milk, Butter} \rightarrow \text{Bread}$$
   - Similar metrics as above, showing strong relationships.
3. $$\text{Bread, Butter} \rightarrow \text{Milk}$$
   - High confidence but slightly weaker lift, still actionable.

Two Items Rules

If desired, association rules at the 1-item values of X could be specified. Consider the rule: Milk → Bread.
Out of total 12 transactions Milk occurs 9 times while (Milk, Bread) occurs 7 times.
The rule has a support level of 7/12 (or 58 percent) and a confidence level of 7/9 (or 77 percent).
Thus, the next valid association rule is as follows:
Milk → Bread {S = 58%, C = 77%}.

2-Item Rules vs 3-Items Rules 2-Item Rules

- **Simplicity**: Easier to interpret and implement in business decisions.
- **Higher Support and Confidence**: More likely to appear frequently in transactions due to fewer constraints.
- **Broad Applicability**: Useful for general trends, such as $\text{Milk} \rightarrow \text{Bread}$, which can inform store layout or promotions.

3-Item Rules

- **Deeper Insights**: Reveal more specific patterns, such as $\text{Milk, Bread} \rightarrow \text{Butter}$, which may not be evident in 2-item rules.
- **Lower Support**: Less likely to occur because all three items must appear together.
- **Targeted Application**: Better suited for niche marketing strategies or tailored product bundles.

Which is Better?

- **2-item rules** are ideal for identifying general trends and implementing broader strategies.
- **3-item rules** are more appropriate for uncovering specific patterns and enabling targeted interventions.

The choice between the two depends on the dataset's characteristics and the business objectives.